

QMOOD Tabanlı Yazılım Kalitesi Analizi ve LLM Destekli Değerlendirme: OpenSearch Anomaly Detection Üzerine Ampirik Bir Çalışma

Yazılım Mimarileri ve Tasarım Yöntemleri Dersi — Dönem Projesi

Abstract—Bu çalışmada, açık kaynak kodlu OpenSearch Anomaly Detection projesinin 12 farklı sürümü üzerinde QMOOD (Quality Model for Object-Oriented Design) modeli kullanılarak sistematik bir yazılım tasarım kalitesi analizi gerçekleştirilmiştir. CK Tool aracılığıyla çıkarılan nesne yönelimli metriklerden (CBO, DIT, WMC, RFC, LCOM) 10 tasarım özelliği hesaplanmış ve Bansiya-Davis formülleriyle 6 kalite niteliği (Reusability, Flexibility, Understandability, Functionality, Extendibility, Effectiveness) elde edilmiştir. Sürüm bazlı evrim analizi, projenin 288 sınıftan 609 sınıfa büyümesi sürecinde coupling’ın monoton artış (%22.1), encapsulation’ın belirgin düşüş (%22.3) ve extendibility’nin sürekli kötüleşme (%26.6) gösterdiğini ortaya koymuştur. Özellikle v2.16.0.0 sürümü, 227 yeni sınıf eklenmesiyle kritik bir kırılma noktası olarak tespit edilmiştir. Çalışmanın ikinci aşamasında, elde edilen metrik verileri ChatGPT (GPT-4), Google Gemini ve Claude olmak üzere üç farklı Büyük Dil Modeli’ne (LLM) sunulmuş ve modellerin yazılım kalitesi değerlendirme yetenekleri karşılaştırılmıştır.

Index Terms—QMOOD, yazılım kalitesi, nesne yönelimli metrikler, yazılım evrimi, teknik borç, büyük dil modelleri, CK metrikleri, OpenSearch

I. GİRİŞ

Yazılım sistemleri, yaşam döngüleri boyunca sürekli değişime uğramakta ve bu değişimler tasarım kalitesini doğrudan etkilemektedir. Lehman’ın yazılım evrimi yasalarına göre [1], bir yazılım sistemi çevresine uyum sağlamak için sürekli değiştirilmeli, ancak bu değişiklikler kontrol edilmediğinde yapısal bozulmaya (architectural erosion) yol açmaktadır.

QMOOD (Quality Model for Object-Oriented Design), Bansiya ve Davis [2] tarafından önerilen ve nesne yönelimli tasarım kalitesini hiyerarşik bir yapıda değerlendiren bir modeldir. Son yıllarda Büyük Dil Modelleri (LLM), yazılım mühendisliğinde kod inceleme, hata tespiti ve kalite değerlendirme gibi alanlarda kullanılmaya başlanmıştır [3].

Bu çalışmada, OpenSearch Anomaly Detection projesinin 12 sürümü QMOOD modeli ile analiz edilmiş ve sonuçlar üç farklı LLM ile değerlendirilmiştir. Çalışmanın katkıları:

- Endüstriyel ölçekte bir Java projesinin 12 sürümlük QMOOD evrim analizi gerçekleştirilmiştir.
- Büyük ölçekli kod eklenmesinin (v2.16) tasarım kalitesine etkisi ampirik olarak gösterilmiştir.
- Üç farklı LLM’in metrik tabanlı kalite değerlendirme yetenekleri karşılaştırılmıştır.
- Prompt mühendisliğinin LLM çıktı kalitesine etkisi incelenmiştir.

II. LİTERATÜR ÖZETİ

A. QMOOD Modeli

QMOOD, Bansiya ve Davis [2] tarafından 2002 yılında önerilen, nesne yönelimli yazılım sistemlerinin tasarım kalitesini hiyerarşik olarak değerlendiren bir modeldir. Model üç katmandan oluşur: (1) nesne yönelimli metrikler (CBO, DIT, WMC, RFC, LCOM, NOM, NOA), (2) 10 tasarım özelliği (Abstraction, Encapsulation, Coupling, Cohesion, Complexity, Design Size, Messaging, Composition, Inheritance, Polymorphism), (3) 6 kalite niteliği (Reusability, Flexibility, Understandability, Functionality, Extendibility, Effectiveness). Model, ISO/IEC 9126 standardıyla uyumludur [4].

B. Chidamber-Kemerer Metrik Seti

CK [5] metrik seti, nesne yönelimli yazılımlar için en yaygın kabul görmüş metrik kümesidir. CBO (Coupling Between Objects), DIT (Depth of Inheritance Tree), WMC (Weighted Methods per Class), RFC (Response for a Class), LCOM (Lack of Cohesion of Methods) ve NOC (Number of Children) metriklerini içerir.

C. Yazılım Evrimi ve Teknik Borç

Lehman [1] yazılım evrimini yöneten sekiz yasa tanımlamıştır. Cunningham [6] tarafından ortaya atılan “teknik borç” metaforu, kısa vadeli tasarım tavizlerinin uzun vadede bakım maliyetlerini artırdığını belirtir.

D. LLM Destekli Yazılım Mühendisliği

Fan vd. [3] ve Hou vd. [7], LLM’lerin yazılım mühendisliği görevlerindeki başarılarını ve sınırlılıklarını kapsamlı biçimde incelemiştir. Al Dallal [8], Padhy vd. [9] ve Singh ve Malhotra [10] QMOOD modelini ampirik olarak doğrulamıştır.

III. YÖNTEM

A. Analiz Edilen Sistem

OpenSearch Anomaly Detection, Amazon OpenSearch platformu üzerinde çalışan, zaman serisi verilerinde anomali tespiti yapan açık kaynak kodlu bir Java eklentisidir. 70’ten fazla release tag’i ile geniş bir evrim sürecine sahiptir.

B. Sürüm Seçim Stratejisi

Kalite değişimlerinin belirgin biçimde gözlemlenmesi amacıyla büyük sıçramalar içeren 12 sürüm seçilmiştir (Tablo I).

TABLE I
ANALİZ EDİLEN SÜRÜMLER VE BOYUT METRİKLERİ

#	Sürüm	Sınıf	Java Dosya	LOC
1	1.0.0.0	288	255	16,903
2	1.1.0.0	327	287	23,354
3	1.3.0.0	351	307	26,338
4	2.0.0.0	351	307	26,363
5	2.2.0.0	356	312	26,604
6	2.4.0.0	356	312	26,610
7	2.7.0.0	362	318	26,932
8	2.10.0.0	362	318	26,930
9	2.13.0.0	364	320	27,041
10	2.16.0.0	591	534	34,392
11	2.19.0.0	609	550	35,613
12	3.0.0.0	609	550	35,612

C. Metrik Çıkarım Aracı

Metrik çıkarımında CK Tool (v0.7.0) [11] kullanılmıştır. Her sürüm için src/main/java dizini analiz edilmiş, test kodları kapsam dışında bırakılmıştır.

D. QMOOD Tasarım Özelliklerinin Hesaplanması

CK Tool çıktılarından QMOOD tasarım özellikleri Tablo II'deki eşlemlerle hesaplanmıştır.

TABLE II
CK TOOL METRİKLERİNDEN QMOOD EŞLEMELERİ

Özellik	Kısalt.	CK Eşlemesi	Hesaplama
Design Size	DSC	–	Toplam sınıf sayısı
Abstraction	ANA	abstractMethodsQty	$\mu(\text{abs}/\text{total})$
Encapsulation	DAM	privateFieldsQty	$\mu(\text{priv}/\text{total})$
Coupling	DCC	cbo	$\mu(\text{CBO})$
Cohesion	CAM	tcc	$\mu(\text{TCC})$
Composition	MOA	totalFieldsQty	$\mu(\text{fields} - \text{static})$
Inheritance	MFA	dit	$P(\text{DIT} > 0)$
Polymorphism	NOP	abstractMethodsQty	$\mu(\text{abs}) + P(\text{DIT} > 0)$
Messaging	CIS	publicMethodsQty	$\mu(\text{publicMethods})$
Complexity	NOM	wmc	$\mu(\text{WMC})$

E. QMOOD Kalite Niteliklerinin Hesaplanması

Kalite nitelikleri, Bansiya ve Davis [2] tarafından tanımlanan doğrusal formüller kullanılarak hesaplanmıştır. DSC değeri $\log_2$ dönüşümüyle normalize edilmiştir.

- $\text{Reusability} = -0.25 \times DCC + 0.25 \times CAM + 0.5 \times CIS + 0.5 \times DSC^*$
- $\text{Flexibility} = 0.25 \times DAM - 0.25 \times DCC + 0.5 \times MOA + 0.5 \times NOP$
- $\text{Understandability} = -0.33 \times ANA + 0.33 \times DAM - 0.33 \times DCC + 0.33 \times CAM - 0.33 \times NOP - 0.33 \times NOM - 0.33 \times DSC^*$
- $\text{Functionality} = 0.12 \times CAM + 0.22 \times NOP + 0.22 \times CIS + 0.22 \times DSC^* + 0.22 \times ANA$
- $\text{Extendibility} = 0.5 \times ANA - 0.5 \times DCC + 0.5 \times MFA + 0.5 \times NOP$
- $\text{Effectiveness} = 0.2 \times ANA + 0.2 \times DAM + 0.2 \times MOA + 0.2 \times MFA + 0.2 \times NOP$

$$DSC^* = \log_2(DSC + 1)$$

F. LLM Destekli Değerlendirme

Üç farklı LLM modeli kullanılmıştır: ChatGPT (GPT-4), Google Gemini ve Claude (Anthropic). Her modele standart bir prompt ile ham metrikler, tasarım özellikleri, kalite nitelikleri ve delta verileri sunulmuştur. Modellerden beş farklı analiz istenmiştir: genel kalite değerlendirmesi, bakım yapılabilirlik analizi, teknik borç tahmini, refactoring önerileri ve mimari kalite yorumları.

IV. ANALİZ SÜRECİ

A. Ham Metrik Sonuçları

Tablo III, 12 sürüme ait ortalama CK metrik değerlerini göstermektedir.

TABLE III
SÜRÜM BAZLI ORTALAMA CK METRİK DEĞERLERİ

Sürüm	Sınıf	μCBO	μDIT	μWMC	μRFC	μLCOM	ΣLOC
1.0.0.0	288	8.58	1.58	11.69	16.16	13.89	16903
1.1.0.0	327	9.24	1.62	14.75	19.05	27.19	23354
1.3.0.0	351	9.61	1.63	15.48	19.90	26.50	26338
2.0.0.0	351	9.62	1.63	15.49	19.91	26.50	26363
2.2.0.0	356	9.67	1.64	15.42	19.91	26.54	26604
2.4.0.0	356	9.67	1.64	15.43	19.91	26.54	26610
2.7.0.0	362	9.75	1.63	15.32	19.79	26.46	26932
2.10.0.0	362	9.75	1.63	15.33	19.83	26.49	26930
2.13.0.0	364	9.78	1.63	15.30	19.77	26.43	27041
2.16.0.0	591	10.48	1.76	12.02	15.41	18.14	34392
2.19.0.0	609	10.47	1.75	12.16	15.50	18.57	35613
3.0.0.0	609	10.47	1.75	12.16	15.52	18.57	35612

Veriler üç belirgin dönemi ortaya koymaktadır: (1) **Hızlı Büyüme** (v1.0–v1.3): Sınıf sayısı %21.9 artmış, LCOM iki katına çıkmıştır. (2) **Stabil Evrim** (v2.0–v2.13): Sınıf sayısı yalnızca %3.7 artmıştır. (3) **Yapısal Dönüşüm** (v2.16–v3.0): Sınıf sayısı %62.4 artmış, ortalama WMC düşmüştür.

V. SONUÇLAR

A. Kalite Nitelikleri

Tablo IV, 12 sürümün QMOOD kalite nitelik değerlerini sunmaktadır.

TABLE IV
QMOOD KALİTE NİTELİKLERİ

Sürüm	Reus.	Flex.	Underst.	Func.	Extend.	Effect.
1.0.0.0	4.147	0.040	−9.572	2.988	−3.286	1.132
1.1.0.0	4.401	−0.041	−10.860	3.177	−3.606	1.167
1.3.0.0	4.409	−0.099	−11.256	3.221	−3.791	1.180
2.0.0.0	4.406	−0.097	−11.265	3.222	−3.800	1.182
2.2.0.0	4.393	−0.114	−11.267	3.222	−3.823	1.181
2.4.0.0	4.393	−0.114	−11.270	3.222	−3.823	1.181
2.7.0.0	4.369	−0.112	−11.267	3.220	−3.864	1.190
2.10.0.0	4.369	−0.111	−11.268	3.220	−3.862	1.190
2.13.0.0	4.361	−0.125	−11.272	3.220	−3.878	1.187
2.16.0.0	4.060	−0.605	−10.744	3.192	−4.163	1.054
2.19.0.0	4.093	−0.617	−10.797	3.204	−4.159	1.047
3.0.0.0	4.093	−0.617	−10.798	3.204	−4.159	1.047

B. Kalite Niteliklerinin Evrimi

Reusability: v1.0'dan v1.3'e artış (4.147 → 4.409), v2.16'da belirgin düşüş (4.361 → 4.060).

Flexibility: v1.0'da pozitif olan tek kalite niteliğidir (0.040). v2.16'da sert düşüş (−0.125 → −0.605).

Understandability: Tüm sürümlerde negatif; ilginç biçimde v2.16'da kısmi iyileşme (−11.272 → −10.744), yeni basit sınıfların ortalamayı düşürmesi nedeniyle.

Functionality: En stabil nitelik; 2.988–3.222 aralığında.

Extendibility: Sürekli kötüleşen trend (−3.286 → −4.159, %26.6 düşüş). En kritik kalite sorunu.

Effectiveness: Nispeten stabil (1.047–1.190). v2.16'da DAM düşüşüyle azalma.

C. Teknik Borç Göstergeleri

- Yüksek LCOM (ort. 18–27): SRP ihlalleri yaygın.
- Max CBO = 217: God Class anti-pattern'i mevcut.
- Max WMC = 318: Aşırı karmaşık sınıflar.
- Max DIT = 7 (sabit): Kalıtım hiyerarşisi kontrol altında.

D. Sürümler Arası Delta Analizi

En büyük kalite değişimleri: v1.0→v1.1'de Understandability −1.288 (hızlı büyüme), v2.13→v2.16'da Flexibility −0.479 (227 yeni sınıf), Extendibility −0.285 (DCC sıçraması), Understandability +0.529 (düşük NOM'lu yeni sınıflar).

VI. TARTIŞMA

A. LLM Değerlendirme Sonuçları

Üç farklı LLM modeline aynı standart prompt ve metrik verileri sunulmuştur.

1) *ChatGPT (GPT-4):* ChatGPT, projenin evrimini iki dönem olarak tanımlamıştır: organik büyüme (v1.0–v2.13) ve kapsamlı yeniden yapılanma (v2.16+). v2.16'yı “teknik borcun türünün değiştiği” bir sürüm olarak tespit etmiştir — monolitik karmaşıklık borcu azalırken entegrasyon borcu artmıştır. Refactoring olarak Facade, Mediator, Event-Driven mekanizmalar ve DDD prensiplerini önermiştir. Mevcut en büyük mimari riskin “yüksek coupling” olduğunu vurgulamıştır.

2) *Google Gemini:* Gemini, en eleştirel analizi sunmuştur. Projenin “katılaşmaya (rigid) başladığını”, v3.0 geçişinin “pazarlama odaklı” olduğunu öne sürmüştür. Max_CBO=214'ü “devasa bir tasarım borcu” olarak tanımlamış, gelecekte özellik ekleme maliyetinin “ekspansiyel olarak artacağını” öngörmüştür. En çarpıcı tespiti: projenin “büyük çamur topu (Big Ball of Mud) evresinden, dağıtılmış ama birbirine dolanmış (Distributed Spaghetti) bir evreye” geçtiğidir. Hexagonal/Clean Architecture önermiştir.

3) *Claude (Anthropic):* Claude, en sistematik ve yapısal analizi sunmuştur. “İki belirgin evreye ayrılan bir gelişim” tespit etmiştir. v2.16'da LCOM düşüşünün “biriken borcun ödenmesinden çok, yeni düşük-LCOM sınıfların ortalamayı aşağı çekmesinden” kaynaklandığını, “eski sınıflardaki kohezyon sorununun yerinde durduğunu” tespit etmiştir. v2.19–v3.0 stabilitesini “mimari donma noktası” olarak yorumlamıştır.

B. LLM Karşılaştırması

Tablo V, üç modelin karşılaştırmalı değerlendirmesini sunmaktadır.

TABLE V
LLM MODELLERİNİN KARŞILAŞTIRMASI

Alan	ChatGPT	Gemini	Claude
Yaklaşım	Dengeli, pratik	Eleştirel, risk odaklı	Sistematik, niçel
v2.16 yorumu	Borç türü değişmiş	Katılaşma	Paradoksal iyileşme
Teknik borç Refactoring	Orta DDD, Facade	Yüksek Hexagonal Arch.	Orta-Yüksek SRP, Composition
v3.0 yorumu	Kararlı durum	İnovasyon kaybı	Donma noktası
Mimari tanı	İyi ama entegre	Big Ball of Mud → Spaghetti	Organik büyüme dönüşüm →

Ortak bulgular: Üç model de (1) coupling artışını en kritik risk olarak tanımlamış, (2) v2.16'yı kırılma noktası olarak nitelendirmiş, (3) LCOM artışının SRP ihlallerine işaret ettiğini belirtmiş, (4) v2.16'da eklenen sınıfların daha basit olduğunu tespit etmiş, (5) God Class ayrıştırmasını öncelikli refactoring olarak belirlemiştir.

Farklılıklar: ChatGPT iyimser ve uygulama odaklı; Gemini en eleştirel (“inovasyon kaybı”); Claude akademik ve metrik temelli. Teknik borç şiddeti konusunda tutarsızlık gözlenmiştir (ChatGPT: orta, Claude: orta-yüksek, Gemini: yüksek).

C. Eleştirel Değerlendirme

LLM çıktıları genel olarak metrik bulgularıyla tutarlıdır. Ancak: (1) bağlam eksikliği nedeniyle refactoring önerileri genel kalmıştır, (2) teknik borç şiddetinde tutarsızlık gözlenmiştir, (3) LLM'ler proje bağlamından bağımsız “standart reçeteler” üretme eğilimindedir, (4) Gemini'nin “pazarlama odaklı” ve Claude'un “bilinçli tasarım kararı” tespitleri spekülatiftir.

D. Geçerlilik Tehditleri

İç geçerlilik: CK Tool dışında çapraz doğrulama yapılmamıştır. **Dış geçerlilik:** Bulgular tek projeye dayanmaktadır. **Yapı geçerliliği:** Bazı QMOOD özelliklerinin proxy metriklerle hesaplanması etkili olabilir. **LLM tekrarlanabilirliği:** Aynı prompt ile farklı yanıtlar üretilebilir.

VII. SONUÇ VE GELECEK ÇALIŞMALAR

Bu çalışmada, OpenSearch Anomaly Detection projesinin 12 sürümü QMOOD modeli kullanılarak analiz edilmiş ve üç farklı LLM ile değerlendirilmiştir. Temel bulgular:

- 1) **Yazılım büyümesi kaliteyi etkiler.** Proje, 288 sınıftan 609 sınıfa (%111 artış) büyümüş; reusability %1.3, flexibility %1644, extendibility %26.6 düşüş göstermiştir.
- 2) **Coupling en kritik kalite sorunudur.** CBO'nun monoton artışı (8.58 → 10.47, %22.1), extendibility, flexibility ve reusability niteliklerini doğrudan olumsuz etkilemektedir.

- 3) **v2.16 yapısal bir kırılma noktasıdır.** 227 yeni sınıf eklenmiş, WMC düşerken DAM %22.0 azalmıştır.
- 4) **LLM’ler metrik tabanlı kalite değerlendirmesinde kullanışlıdır.** Üç model de sürüm trendlerini doğru yorumlamış ve anlamlı öneriler sunmuştur.
- 5) **Prompt mühendisliği LLM çıktı kalitesini belirler.** Ham metriklerin ve spesifik soruların eklenmesi, çıktı derinliğini önemli ölçüde artırmıştır.

Gelecek çalışmalar: (i) v3.1–v3.7 sürümlerinin analizi, (ii) en yüksek CBO/LCOM sınıflarının tekil analizi, (iii) Designite/SonarQube ile çapraz doğrulama, (iv) LLM’lere kaynak kod verilerek spesifik refactoring önerileri, (v) başka bir açık kaynak projeye karşılaştırma, (vi) SQALE modeli ile teknik borç tahmini.

REFERENCES

- [1] M. M. Lehman, “Laws of software evolution revisited,” in *Proc. 5th European Workshop on Software Process Technology (EWSPT)*, 1996, pp. 108–124.
- [2] J. Bansiya and C. G. Davis, “A hierarchical model for object-oriented design quality assessment,” *IEEE Trans. Softw. Eng.*, vol. 28, no. 1, pp. 4–17, Jan. 2002.
- [3] A. Fan *et al.*, “Large language models for software engineering: Survey and open problems,” *arXiv preprint arXiv:2310.03533*, 2023.
- [4] ISO/IEC 9126-1:2001, “Software engineering — Product quality — Part 1: Quality model,” International Organization for Standardization, 2001.
- [5] S. R. Chidamber and C. F. Kemerer, “A metrics suite for object-oriented design,” *IEEE Trans. Softw. Eng.*, vol. 20, no. 6, pp. 476–493, Jun. 1994.
- [6] W. Cunningham, “The WyCash portfolio management system,” in *Proc. OOPSLA ’92 Experience Report*, 1992.
- [7] X. Hou *et al.*, “Large language models for software engineering: A systematic literature review,” *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 8, pp. 1–79, 2024.
- [8] J. Al Dallal, “Object-oriented class maintainability prediction using internal quality attributes,” *Inf. Softw. Technol.*, vol. 55, no. 11, pp. 2028–2048, 2013.
- [9] N. Padhy, S. Satapathy, and R. P. Singh, “State-of-the-art object-oriented metrics and its reusability: A decade review,” in *Smart Computing and Informatics*, Springer, 2018, pp. 431–441.
- [10] G. Singh and R. Malhotra, “An empirical investigation of the effect of object-oriented design metrics on quality,” *Softw. Qual. J.*, vol. 30, no. 4, pp. 983–1012, 2022.
- [11] M. Aniche, “CK: Calculate Chidamber and Kemerer and many other metrics for Java projects,” GitHub, 2015. [Online]. Available: <https://github.com/mauricioaniche/ck>
- [12] R. C. Martin, *Agile Software Development, Principles, Patterns, and Practices*. Upper Saddle River, NJ: Prentice Hall, 2003.
- [13] M. Fowler, *Refactoring: Improving the Design of Existing Code*, 2nd ed. Boston, MA: Addison-Wesley, 2018.
- [14] OpenSearch Project, “anomaly-detection,” GitHub, 2024. [Online]. Available: <https://github.com/opensearch-project/anomaly-detection>